

Aulas 17-19: Projeto Full-Stack (Sistema de E-Commerce)

Duração: 12 horas (3 aulas de 4 horas)

Visão Geral do Projeto

Objetivo: Construir um e-commerce completo com:

- **Frontend:** Angular + Tailwind CSS
- **Backend:** Spring Boot + MySQL
- **Funcionalidades:** Autenticação JWT, CRUD de produtos, carrinho de compras, checkout simulado

Aula 17: Backend (Spring Boot)

Parte 1: Configuração Inicial (1h)

1. Setup do Projeto:

```
bash
```

```
spring init --dependencies=web,data-jpa,mysql,security,lombok ecommerce-backend
```

2. Entidades Principais:

```
java
```

```
@Entity
@Data @NoArgsConstructor @AllArgsConstructor
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private double price;
    private String imageUrl;
}
```

```
@Entity
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String username;
    private String password; // Hash com BCrypt
}
```

```
@Enumerated(EnumType.STRING)
private Role role;
}
```

Parte 2: API REST (2h)

1. Controllers:

java

```
@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {
    private final ProductRepository repository;

    @GetMapping
    public List<Product> getAll() {
        return repository.findAll();
    }
}
```

2. Spring Security + JWT:

- Configurar `JwtAuthFilter` e `SecurityConfig` (baseado na Aula 11).

Parte 3: Testes (1h)

- Testes de integração para `ProductController` com `@WebMvcTest`.

Aula 18: Frontend (Angular)

Parte 1: Estrutura Inicial (1h)

1. Setup:

bash

```
ng new ecommerce-frontend --routing --style=scss
npm install tailwindcss daisyui @angular/material
```

2. Módulos Principais:

typescript

```
@NgModule({
  imports: [
```

```

    HttpClientModule,
    RouterModule.forRoot([
      { path: '', component: HomeComponent },
      { path: 'products', component: ProductListComponent }
    ])
  ]
})
export class AppModule {}

```

Parte 2: Páginas (2h)

1. Lista de Produtos:

html

```

<!-- product-list.component.html -->
<div class="grid grid-cols-1 md:grid-cols-3 gap-4">
  <div *ngFor="let product of products" class="card bg-base-100 shadow-xl">
    <img [src]="product.imageUrl" alt="Product">
    <div class="card-body">
      <h2 class="card-title">{{ product.name }}</h2>
      <p>{{ product.price | currency }}</p>
      <button (click)="addToCart(product)" class="btn btn-primary">Add to Cart</button>
    </div>
  </div>
</div>

```

2. Carrinho:

- o Serviço com BehaviorSubject para gerenciar estado.

Parte 3: Integração com API (1h)

- Chamadas HTTP com interceptors JWT.

Aula 19: Integração e Features Avançadas

Parte 1: Checkout (2h)

1. Formulário com Validação:

typescript

```

checkoutForm = new FormGroup({
  name: new FormControl('', Validators.required),
  cardNumber: new FormControl('', [Validators.required, Validators.pattern('[0-9]{1

```

```
6}')]])  
});
```

2. Simulação de Pagamento:

- Serviço fake que retorna `Observable` com delay.

Parte 2: Deploy (1h)

1. Backend no Heroku:

```
bash  
  
heroku create  
git push heroku main
```

2. Frontend na Vercel:

```
bash  
  
npm run build  
vercel --prod
```

Parte 3: Testes Finais (1h)

- Testes E2E com Cypress.

Tarefa Final

Implementar uma das features extras:

- Sistema de avaliações (stars)
- Filtros por categoria
- Histórico de pedidos

Recursos

- [Repositório Modelo](#)
- [Design no Figma](#)

plaintext

- ✓ Backend: API REST funcional
- ✓ Frontend: UI responsiva

✓ Integração completa

https://miro.medium.com/v2/resize:fit:1400/1*VYz9sV5uKrFg3j0fu6vW3A.png (*Fluxo do E-Commerce*) 🚀